

PROJECT WALKTHROUGH

Galatiq : Invoice Processing

How the multi-agent system works — what it is,
why I built it this way, and what every part does.

Powered by Google Gemini · 4 agents · CLI + interactive dashboard

16 September 2026

1 · THE PROBLEM & THE SOLUTION

What is this, really?

So, let me start with the problem, because it's a real one. Acme Corp — a mid-size manufacturer — is losing around **two million dollars a year** just processing invoices by hand. Invoices land as messy PDFs and emails, and a person has to read each one, type the numbers in, check them against inventory, chase a VP for sign-off, and finally pay. The outcome is a **30% error rate** and **five-day** delays.

What I built is a working prototype that takes that whole chain and hands it to **four cooperating agents** that cooperate like a small accounts-payable team. You give it an invoice — in almost any format — and it reads it, checks it, decides, and either pays it or blocks it. In seconds. And the reasoning behind every decision is right there on screen, so nothing is a black box.

In one line: drop in an invoice, and the system reads it, checks it, reasons about it, and pays or blocks it — catching errors and fraud before any money moves.

2 · THE BIG PICTURE

How it fits together

Here's the shape of it. A single shared record — the invoice's *state* — travels through four agents. Each one reads it, does its one job, and writes its findings back. After the approval step, a router sends the invoice down one of three paths.



Under the hood this is a small **state graph** — the same idea as a framework like LangGraph (typed state, named steps, conditional routing) — but I wrote it in a few clean lines so it's easy to read and runs anywhere. If we wanted to scale it later, moving to real LangGraph is mechanical; the shapes already match.

3 · THE FOUR AGENTS

Who does what — and why it's split this way

Before the details — why four separate agents instead of one big script? Two reasons. First, **separation of concerns**: each agent does exactly one job, so I can test it on its own and swap it out without touching the rest. Second, it **mirrors a real AP team** — one person extracts, one validates, a VP approves, finance pays — which makes the whole design easy to explain to the business.

Ingestion

It reads the raw file — TXT, JSON, CSV, XML, or PDF — and pulls out the vendor, amount, line items and due date. Real invoices are messy, so it quietly fixes typos (INVOCE, Vndr) and scan noise (2026 → 2026). It also scores its own **confidence**, and repairs recoverable gaps — like recomputing a missing total — instead of just failing.

Validation

This one checks every item against the inventory database. It flags items that exceed stock, items that aren't in the catalogue, and negative quantities. On top of that it computes a **fraud score** from independent signals: a zero-stock 'trap' item, high-pressure language like 'pay immediately / wire transfer', or an impossible due date such as 'yesterday'.

Approval

This is the brain — it acts as the approver, and its reasoning is **powered by Google Gemini**.

Anything **\$10,000 or more** gets extra scrutiny. It makes a first decision, then runs a **self-review pass** — a second pass that critiques its own call and can overturn it. For example, a large but perfectly clean invoice is first held for review, then cleared to pay once the self-review confirms there's no real risk.

Payment

The last step. If the invoice is approved, it calls the payment API and records a reference. If it's escalated, it parks it for review. If it's rejected, it logs the reason and pays nothing.

Three outcomes, and the logic

■ PAID	Clean, in stock, and within the threshold (or a high-value invoice cleared by the self-review pass). Paid automatically.
■ ESCALATED	Flagged for review: a stock overrun, an unknown item, a foreign currency, or low extraction confidence. Held, not paid.
■ REJECTED	Fraud signals or blocking data errors (missing vendor, negative amount). No payment attempted.

Why three outcomes, not just yes or no?

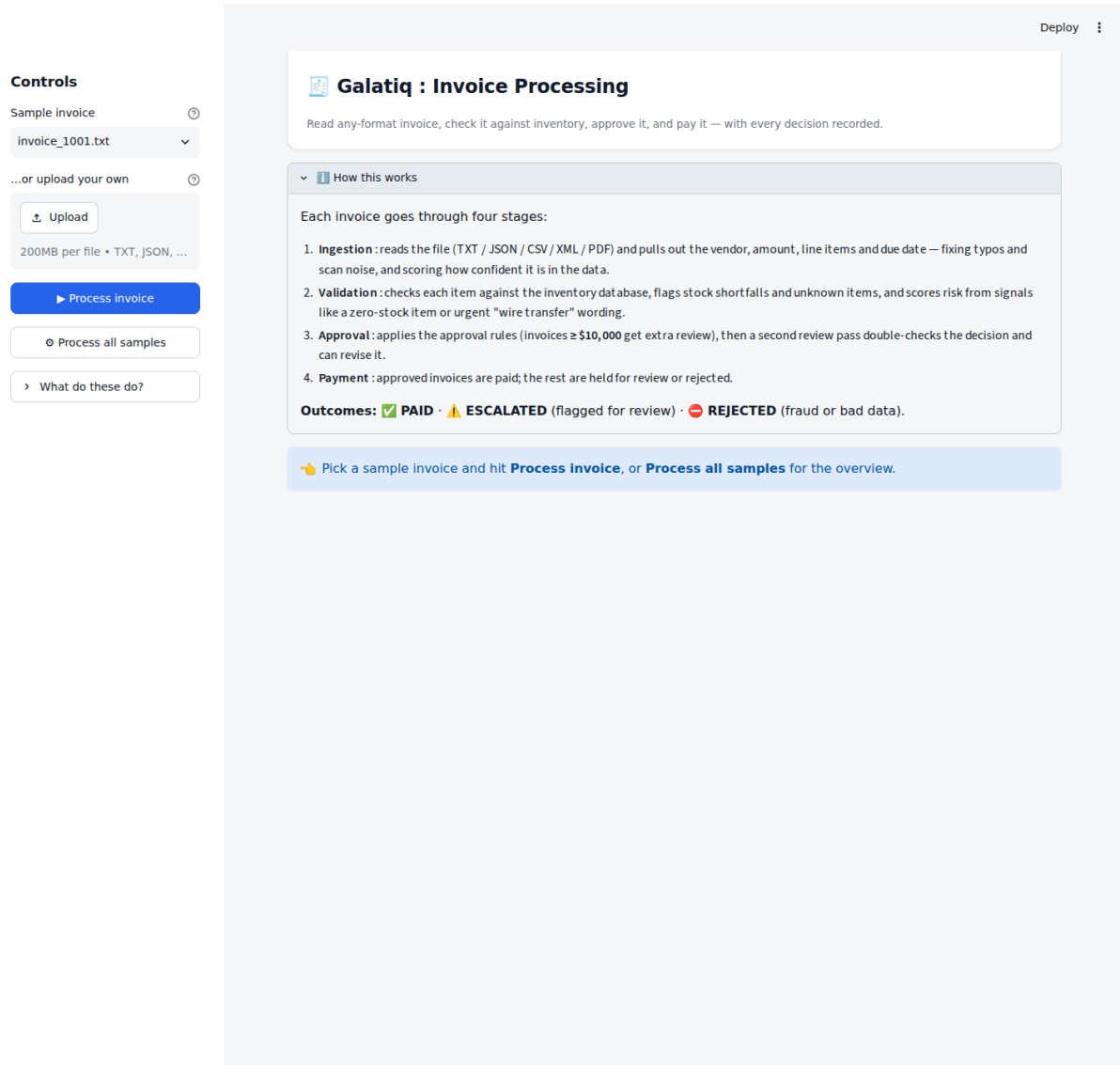
Because in real accounts-payable, most problems aren't fraud — they're a typo, or an item a buyer needs to confirm. Rejecting those outright would annoy good vendors. So I **escalate** them to a reviewer and save hard **rejection** for genuine fraud and bad data. Internally, findings are sorted into **errors** (block), **review flags** (escalate) and **warnings** (informational).

Self-correction in action: INV-1017 is a \$12,500 invoice — above the scrutiny bar — but completely clean. The first pass says ESCALATE; the self-review pass looks again, confirms there's no real risk, and changes the decision to APPROVE. That's the agent checking its own work.

5 · THE DASHBOARD

Every control, and how to drive it

You launch it with `streamlit run app.py` and it opens in the browser. Here's the starting screen:



The landing screen — header, the built-in ‘How this works’ panel, and the controls on the left.

The controls (left sidebar):

- **Sample invoice** : a dropdown of the provided test invoices — a mix of clean, messy and fraudulent ones.
- **Upload** : drag in your own invoice file (TXT / JSON / CSV / XML / PDF) instead of a sample.
- **Process invoice** : run the selected or uploaded invoice through all four agents.

- **Process all samples** : run the whole batch and open the interactive overview.
- **How this works** : a built-in explainer, so anyone can understand the screen without a manual.

The batch overview — and it's clickable

When you run the whole batch, you get the view a finance leader actually cares about: how many paid, how many went for review, how many were rejected, and the dollar value held back from auto-pay. Every block is interactive.

The screenshot displays the Galatiq Invoice Processing interface. On the left is a 'Controls' sidebar with options to upload a sample invoice or a custom file, and buttons to process the invoice or all samples. The main area is titled 'Galatiq : Invoice Processing' and includes a 'Batch overview' section with five summary cards: Invoices (17), Paid (8), Escalated (7), Rejected (2), and Held from auto-pay (\$185K). Below these are buttons to view each category. The 'All invoices' section lists seven individual invoices with their IDs, company names, amounts, statuses, and file formats, each with an 'Open' button.

Batch overview				
Invoices 17	Paid 8	Escalated 7	Rejected 2	Held from auto-pay \$185K

All invoices				
INV-1001 · Widgets Inc. TXT	5,000 USD		Open →	
INV-1002 · Gadgets Co. TXT	15,000 USD		Open →	
INV-1003 · Fraudster LLC TXT	100,000 USD		Open →	
INV-1004 · Precision Parts Ltd. JSON	1,890 USD		Open →	
INV-1005 · Global Supply Chain Partners JSON	15,225 USD		Open →	
INV-1006 · Acme Industrial Supplies CSV	2,750 USD		Open →	
INV-1007 · MegaWidgets Corp CSV	15,525 USD		Open →	

Batch overview — click 'View paid / escalated / rejected' to drill into just those invoices.

Drilling in:

- Click an outcome (say **View rejected**) and the list filters to exactly those invoices — no scrolling through the rest.
- Every invoice row has an **Open** → button that opens its full breakdown.
- From a breakdown, ← **Back to summary** returns you to the overview, right where you were.

One invoice, in full

Opening any invoice shows the whole journey as blocks — the stages it went through, the decision, the key numbers, and what each agent found:

The screenshot displays the Galatq Invoice Processing interface. On the left, a 'Controls' sidebar includes a 'Sample invoice' dropdown set to 'invoice_1001.txt', an 'Upload' button, and a 'Process invoice' button. The main area shows the invoice title 'INV-1001 : Widgets Inc.' and a progress bar with four stages: Ingestion, Validation, Approval, and Payment. The 'Decision' section shows a 'PAID' status. Key metrics include 'Amount : USD 5,000', 'Confidence 100%', and 'Fraud score 0'. The workflow is detailed in four blocks: 1. Ingestion (details read from the file), 2. Validation (checked against inventory, no issues), 3. Approval (reasoned by Gemini, approving payment), and 4. Payment (approved invoices are paid via the payment API).

Controls

Sample invoice
invoice_1001.txt

...or upload your own

Upload

200MB per file • TXT, JSON, ...

Process invoice

Process all samples

What do these do?

Galatq : Invoice Processing

Read any-format invoice, check it against inventory, approve it, and pay it — with every decision recorded.

How this works

Back

INV-1001 : Widgets Inc.

Ingestion → Validation → Approval → Payment

Decision

PAID

Amount : USD
5,000

Confidence
100%

Fraud score
0

1 • Ingestion

Details read from the file (typos and scan noise fixed).

{...}

item	quantity	unit_price	amount	note
WidgetA	10	250	2500	
WidgetB	5	500	2500	

2 • Validation

Checked against inventory. Errors block; flags go to review; warnings are soft.

item	requested	stock	status
WidgetA	10	15	ok
WidgetB	5	10	ok

No validation issues.

3 • Approval

The decision, with a second review pass that can revise the first call.

Reasoned by Gemini (gemini-3.5-flash)

Approving payment. Clean invoice 5,000.00 USD, under 10,000, all items valid and in stock. Reflection — Draft decision is consistent with the evidence; no change.

4 • Payment

Approved invoices are paid via the payment API; others are held or logged.

Paid 5,000.00 USD to Widgets Inc. (ref PAY-93811F5306).

A single invoice (the \$100K fraud case) — stage chips, the decision, and a block per agent.

- **Stage chips** : the path the invoice took; the last chip is coloured by the outcome.
- **Decision • Amount • Confidence • Fraud score** : the numbers that most affect the decision.
- **Block 1 — Ingestion** : the fields extracted and the line items found.
- **Block 2 — Validation** : the inventory check per item, plus any errors, fraud signals or warnings.
- **Block 3 — Approval** : the approval reasoning, and a note if the self-review pass changed the decision.

- **Block 4 — Payment** : the payment reference, the hold notice, or the rejection reason.

6 · READING THE RESULTS

What the numbers mean

- **Confidence** : how sure the Ingestion agent is about the data it extracted. Low confidence pushes an invoice to review.
- **Fraud score** : adds up independent red flags. A high score forces a rejection regardless of the amount.
- **Held from auto-pay** : the total dollar value the agents stopped from being paid automatically — the risk the system catches before money moves. That's the number I'd put in front of a CFO.

7 · SAMPLE RESULTS

All 17 test invoices, and why

Invoice	Fmt	Amount	Outcome	Why
INV-1001	TXT	5,000 USD	PAID	Clean, in stock, under threshold
INV-1002	TXT	15,000 USD	ESCALATED	Quantity for 'GadgetX' (20) exceeds available stock (5).
INV-1003	TXT	100,000 USD	REJECTED	Fraud : zero-stock item requested: FakeItem
INV-1004	JSON	1,890 USD	PAID	Clean, in stock, under threshold
INV-1005	JSON	15,225 USD	ESCALATED	Quantity for 'GadgetX' (8) exceeds available stock (5).
INV-1006	CSV	2,750 USD	PAID	Clean, in stock, under threshold
INV-1007	CSV	15,525 USD	ESCALATED	Quantity for 'WidgetA' (20) exceeds available stock (15).
INV-1008	TXT	9,900 USD	ESCALATED	Unknown item not in inventory: 'SuperGizmo'.
INV-1009	JSON	-250 USD	REJECTED	Missing vendor name — cannot verify payee.
INV-1010	TXT	6,700 USD	PAID	Clean (minor note)
INV-1011	TXT	3,000 USD	PAID	Clean, in stock, under threshold
INV-1012	TXT	9,500 USD	PAID	Clean, in stock, under threshold
INV-1013	JSON	22,563 USD	ESCALATED	Quantity for 'WidgetA' (22) exceeds available stock (15).
INV-1014	XML	4,125 EUR	ESCALATED	Invoice is in EUR; needs FX conversion + sign-off before p...

Invoice	Fmt	Amount	Outcome	Why
INV-1015	CSV	6,500 USD	PAID	Clean, in stock, under threshold
INV-1016	JSON	3,233 USD	ESCALATED	Unknown item not in inventory: 'WidgetC'.
INV-1017	JSON	12,500 USD	PAID	Clean, in stock, under threshold

Generated live by running the pipeline on the sample set — these are the system's actual decisions.

8 · RUN IT LOCALLY

Setup on your machine

```
python -m pip install -r requirements.txt
python setup_db.py # build the inventory DB (once)

python main.py --all # process everything in the terminal
python -m streamlit run app.py # the interactive dashboard
python report.py # generate the HTML report
```

Needs Python 3.10+. Set your API key first (next section) to power the reasoning.

9 · POWERING IT WITH GROK

Where the API key goes

The approval agent's reasoning is powered by Google Gemini. You add your key in one place — either as an environment variable or directly in the config file:

- `config.py`, line 8 : the provider — already set to "gemini".
- `config.py`, line 9 : paste your key — `LLM_API_KEY_DEFAULT = "xai-..."`.
- Or set the environment variables `LLM_PROVIDER` and `LLM_API_KEY` and leave the file untouched.

The system is resilient by design: if the model is briefly unreachable, the agent falls back to its deterministic rules so an API hiccup never blocks a payment run.

10 · WHY I BUILT IT THIS WAY

The decisions behind it

- **Four agents, not one call** : a single black-box call is hard to test and hard to trust with money. Splitting it makes every step inspectable and unit-tested.
- **Three outcomes, not two** : escalate vendor mistakes to a reviewer; hard-reject only genuine fraud and bad data.
- **A self-review pass** : the approval agent checks its own decision, which catches both over-lenient and over-harsh calls.
- **An interactive dashboard** : the batch view isn't a static report — you click an outcome and drill straight into those invoices, then open any one in full.

- **Resilience over fragility** : deterministic guardrails around the model mean a network blip never stops a payment run.
- **Scoped cuts** : real image OCR and live currency conversion are deliberately out of scope for the MVP (foreign-currency invoices are escalated instead).

11 · QUALITY

Tests & tech

Python, SQLite, pdfplumber, Streamlit, and Google Gemini for the reasoning. **31 automated tests** cover parsing (every format plus typo/OCR cases), each validation rule, the approval and self-review logic, and an end-to-end check that all 17 sample invoices reach the decision I expect — so I can change something and instantly know if I broke a case.